

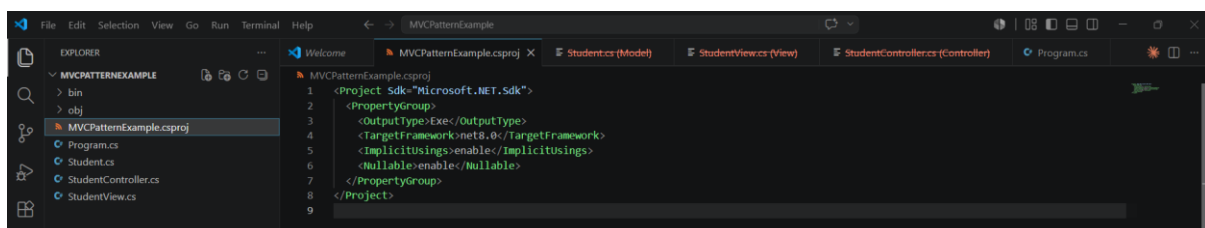
# Hands-On Report

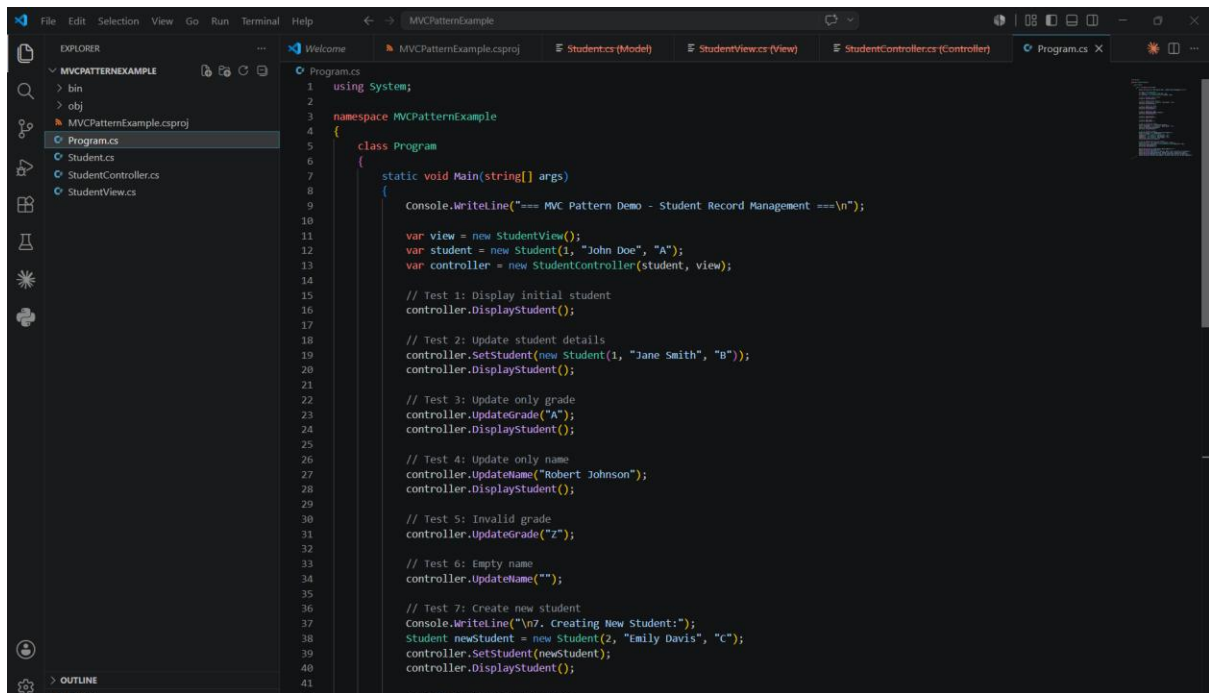
## Exercise 10: Implementing the MVC Pattern

In this exercise, I implemented the Model-View-Controller (MVC) Pattern in a .NET application by developing a project named **MVCPatternExample**. I created a **Student** model to store student details, a **StudentView** class to display the information, and a **StudentController** class to manage communication between the model and the view.

After running the application, I tested the system by adding, updating, and displaying student records through the controller. The results confirmed that the data was correctly managed and presented in the user interface. This practical exercise helped me understand how the MVC Pattern separates data, business logic, and presentation, resulting in a well-structured, scalable, and easy-to-maintain .NET application.

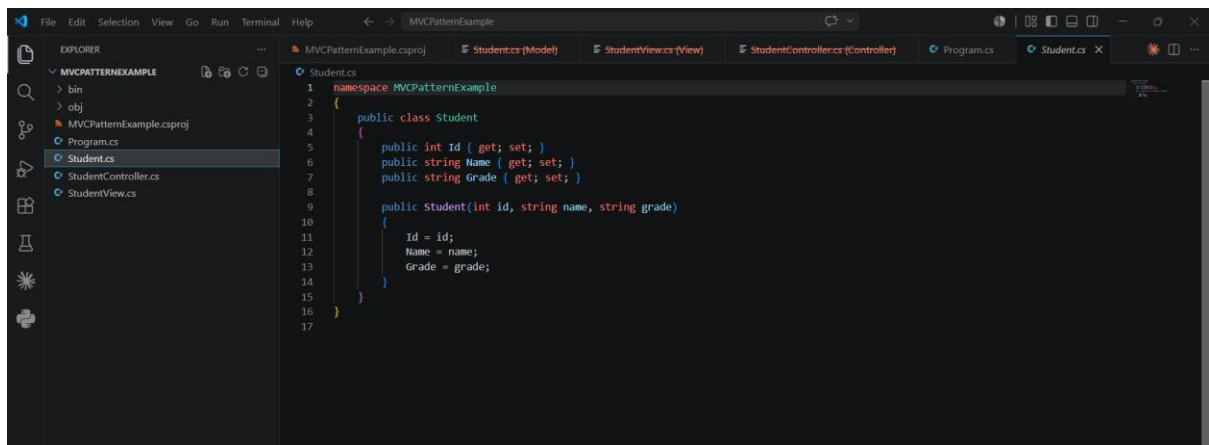
### Coding:





This screenshot shows the Visual Studio IDE with the 'Program.cs' file open. The Explorer pane on the left shows the project structure for 'MVCPATTERNEXAMPLE', including 'bin', 'obj', 'MVCPatternExample.csproj', and source files 'Program.cs', 'Student.cs', 'StudentController.cs', and 'StudentView.cs'. The 'Program.cs' file contains the following code:

```
1 using System;
2
3 namespace MVCPatternExample
4 {
5     class Program
6     {
7         static void Main(string[] args)
8         {
9             Console.WriteLine("=== MVC Pattern Demo - Student Record Management ===\n");
10
11             var view = new StudentView();
12             var student = new Student(1, "John Doe", "A");
13             var controller = new StudentController(student, view);
14
15             // Test 1: Display initial student
16             controller.DisplayStudent();
17
18             // Test 2: Update student details
19             controller.SetStudent(new Student(1, "Jane Smith", "B"));
20             controller.DisplayStudent();
21
22             // Test 3: Update only grade
23             controller.UpdateGrade("A");
24             controller.DisplayStudent();
25
26             // Test 4: Update only name
27             controller.UpdateName("Robert Johnson");
28             controller.DisplayStudent();
29
30             // Test 5: Invalid grade
31             controller.UpdateGrade("Z");
32
33             // Test 6: Empty name
34             controller.UpdateName("");
35
36             // Test 7: Create new student
37             Console.WriteLine("\n7. Creating New Student:");
38             Student newStudent = new Student(2, "Emily Davis", "C");
39             controller.SetStudent(newStudent);
40             controller.DisplayStudent();
41
42             // Test 8: Delete student
43             controller.DeleteStudent(1);
44             controller.DisplayStudent();
45         }
46     }
47 }
```



This screenshot shows the Visual Studio IDE with the 'Student.cs' file open. The Explorer pane on the left shows the project structure for 'MVCPATTERNEXAMPLE', including 'bin', 'obj', 'MVCPatternExample.csproj', and source files 'Program.cs', 'Student.cs', 'StudentController.cs', and 'StudentView.cs'. The 'Student.cs' file contains the following code:

```
1 namespace MVCPatternExample
2 {
3     public class Student
4     {
5         public int Id { get; set; }
6         public string Name { get; set; }
7         public string Grade { get; set; }
8
9         public Student(int id, string name, string grade)
10         {
11             Id = id;
12             Name = name;
13             Grade = grade;
14         }
15     }
16 }
17
```

```
1 using System;
2
3 namespace MVCPatternExample
4 {
5     public class StudentController
6     {
7         private Student _student;
8         private readonly StudentView _view;
9
10        public StudentController(Student student, StudentView view)
11        {
12            _student = student;
13            _view = view;
14        }
15
16        public void SetStudent(Student student)
17        {
18            _student = student;
19            Console.WriteLine("SUCCESS: Student details updated successfully");
20        }
21
22        public void UpdateName(string name)
23        {
24            if (string.IsNullOrWhiteSpace(name))
25            {
26                Console.WriteLine("ERROR: Name cannot be empty");
27                return;
28            }
29            _student.Name = name;
30            Console.WriteLine($"SUCCESS: Name updated to {name}");
31        }
32
33        public void UpdateGrade(string grade)
34        {
35            string[] validGrades = { "A", "B", "C", "D", "F" };
36            if (Array.IndexOf(validGrades, grade) == -1)
37            {
38                Console.WriteLine("ERROR: Invalid grade. Must be A, B, C, D, or F");
39                return;
40            }
41            _student.Grade = grade;
42            Console.WriteLine($"SUCCESS: Grade updated to {grade}");
43        }
44    }
45 }
```

```
1 using System;
2
3 namespace MVCPatternExample
4 {
5     public class StudentView
6     {
7         public void DisplayStudent(Student student)
8         {
9             Console.WriteLine("\n== Student Details ==");
10            Console.WriteLine($"Student ID: {student.Id}");
11            Console.WriteLine($"Student Name: {student.Name}");
12            Console.WriteLine($"Student Grade: {student.Grade}");
13        }
14
15        public void DisplayStudentList(Student[] students)
16        {
17            Console.WriteLine("\n== Student list ==");
18            Console.WriteLine($"Total Students: {students.Length}");
19            foreach (var student in students)
20            {
21                Console.WriteLine($"ID: {student.Id}, Name: {student.Name}, Grade: {student.Grade}");
22            }
23        }
24    }
25 }
26
```

